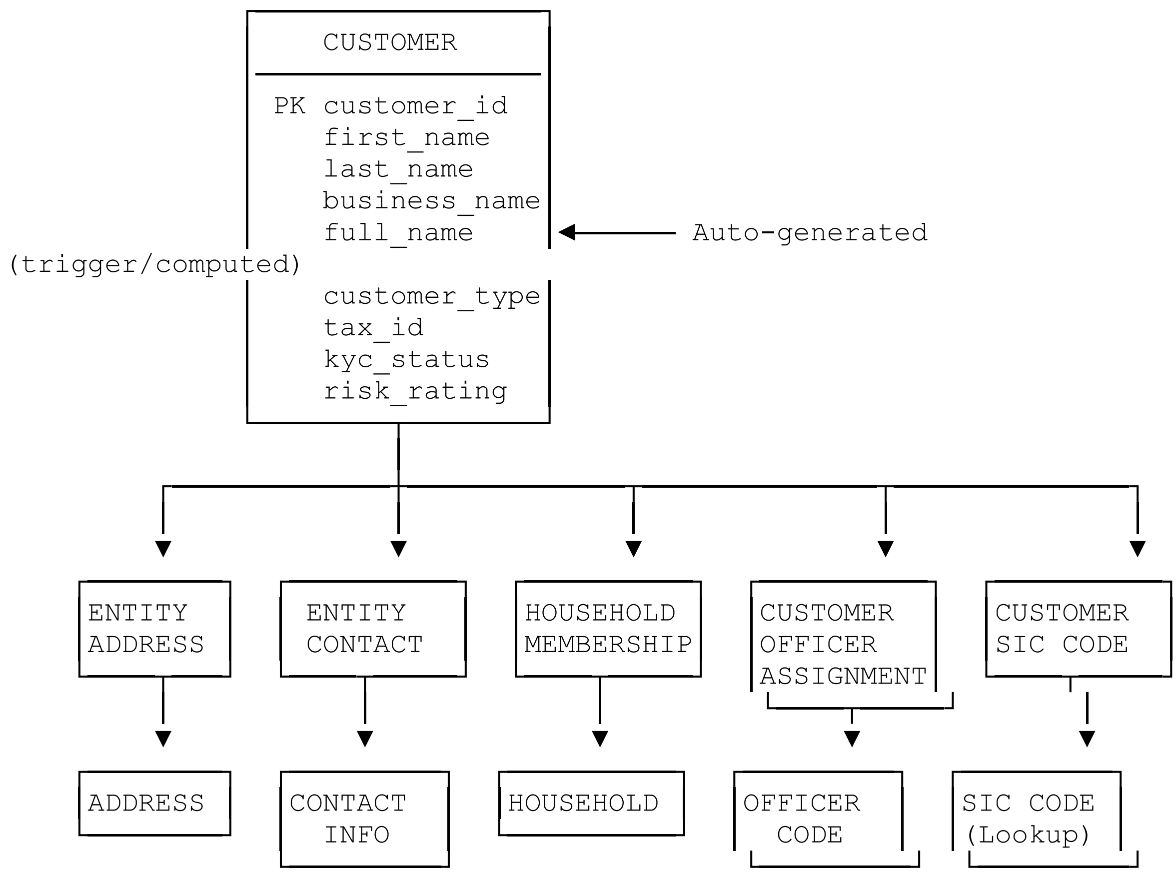
ClientIQ - Entity Relationship Diagram

Version: 1.0
Last Updated: October 22, 2025
Database Engines: PostgreSQL 14+ | SQL Server 2019+

Complete Entity Relationship Overview

CORE BANKING ENTITIES



```
              ACCOUNT
       ---------------------
        PK account_id
           account_num
           account_type   <------  checking, savings, loan, etc.
           balance
           credit_limit
        FK branch_id
        FK sic_code

    ACCOUNT      FINANCIAL      DEBIT      ENTITY
    OWNERSHIP    TRANSACTION    CARD       ADDRESS

    CUSTOMER
```

FINANCIAL TRANSACTION FLOW

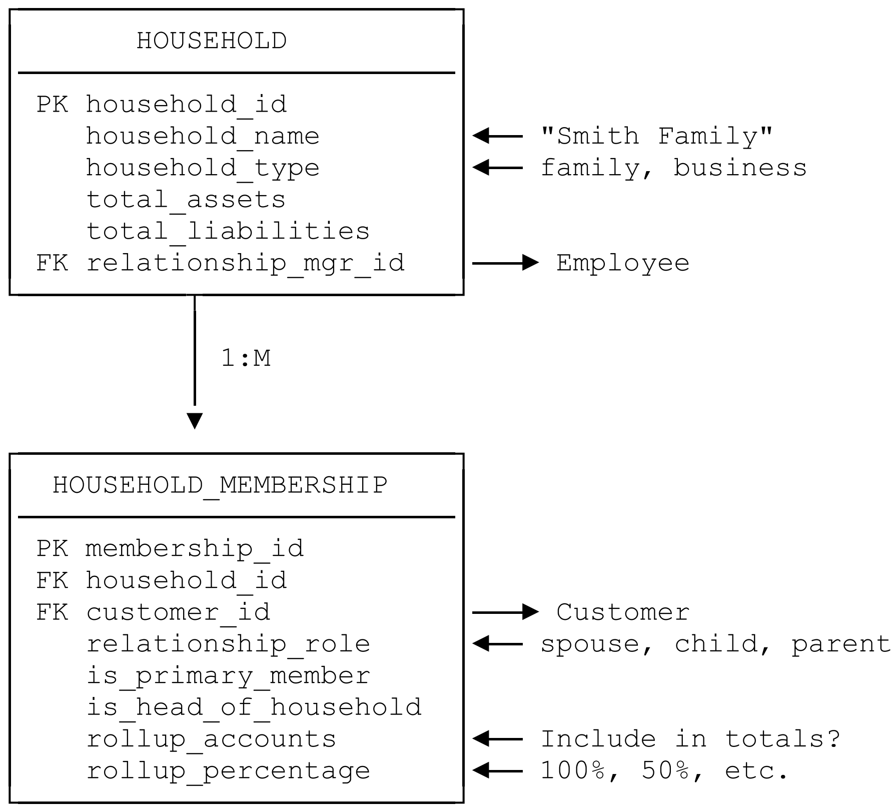
```
           FINANCIAL_TRANSACTION (Central Hub)
  --------------------------------------------------------------------
  PK transaction_id
  FK account_id                     --------> Links to Account
  FK debit_card_id (optional)-------> Links to Debit Card
  FK category_id                    --------> Transaction Category
  FK counterparty_account_id        --------> Transfer Counterparty

     amount
     transaction_date
     posting_date
     merchant_name
     merchant_category_code
     ledger_balance_after          <------  Balance tracking
     available_balance_after       <------  Balance tracking
     source_system                 <------  jack_henry, silverlake
     source_transaction_id
     transfer_group_id             <------  Groups related transfers
```

DEBIT CARD ARCHITECTURE

```
          ┌──────────────────────────────┐
          │ DEBIT_CARD_LIMIT_PROFILE     │
          ├──────────────────────────────┤
          │ PK profile_id                │
          │    profile_name              │ ◄── "Standard Personal"
          │    daily_purchase_limit      │ ◄── $2,500
          │    daily_atm_limit           │ ◄── $500
          │    single_txn_limit          │ ◄── $1,000
          │    monthly_limit             │ ◄── $10,000
          └──────────────────────────────┘
                    │
                    │ FK limit_profile_id
                    ▼
          ┌──────────────────────────────┐
          │      DEBIT_CARD              │
          ├──────────────────────────────┤
          │ PK card_id                   │
          │ FK account_id                │ ◄── MUST be checking
account
          │ FK limit_profile_id          │     (enforced by trigger)
          │    card_type                 │ ◄── primary, companion
          │    card_status               │ ◄── active, suspended,
closed
          │    last_four_digits          │ ◄── PCI compliant (only
last 4)
          │    card_brand                │ ◄── Visa, Mastercard
          │    expiry_month              │
          │    expiry_year               │
          │    cardholder_name           │
          │    jack_henry_card_id        │ ◄── Core banking
reference
          └──────────────────────────────┘
                    │
                    │ FK debit_card_id (optional)
                    ▼
          ┌──────────────────────────────┐
          │   FINANCIAL_TRANSACTION      │
          ├──────────────────────────────┤
          │    debit_card_id             │ ◄── Links card to
transaction
          │    merchant_name             │
          │    merchant_category_code    │
          │    amount                    │
```

HOUSEHOLD RELATIONSHIP MODEL



BRANCH & EMPLOYEE MODEL

```
BRANCH                      EMPLOYEE
-----------------           -----------------
PK branch_id                PK employee_id
   branch_code                 emp_number
   branch_name                 first_name
   branch_type                 last_name
FK address_id                  officer_code

            | M:M
            v

EMPLOYEE_BRANCH
---------------------
PK emp_branch_id
FK employee_id
FK branch_id
   assignment_role    <--  "Branch Manager"
   is_primary         <--  Primary branch flag
   is_active
```

ENTITY-ATTRIBUTE-VALUE PATTERNS

```
ENTITY_ADDRESS (Polymorphic)
----------------------------------------------------------------
PK entity_address_id
   entity_type          <--  "customer", "branch", "employee"
   entity_id            <--  References ANY entity
FK address_id           -->  ADDRESS table
   address_purpose      <--  "primary", "mailing", "billing"
   is_current
```

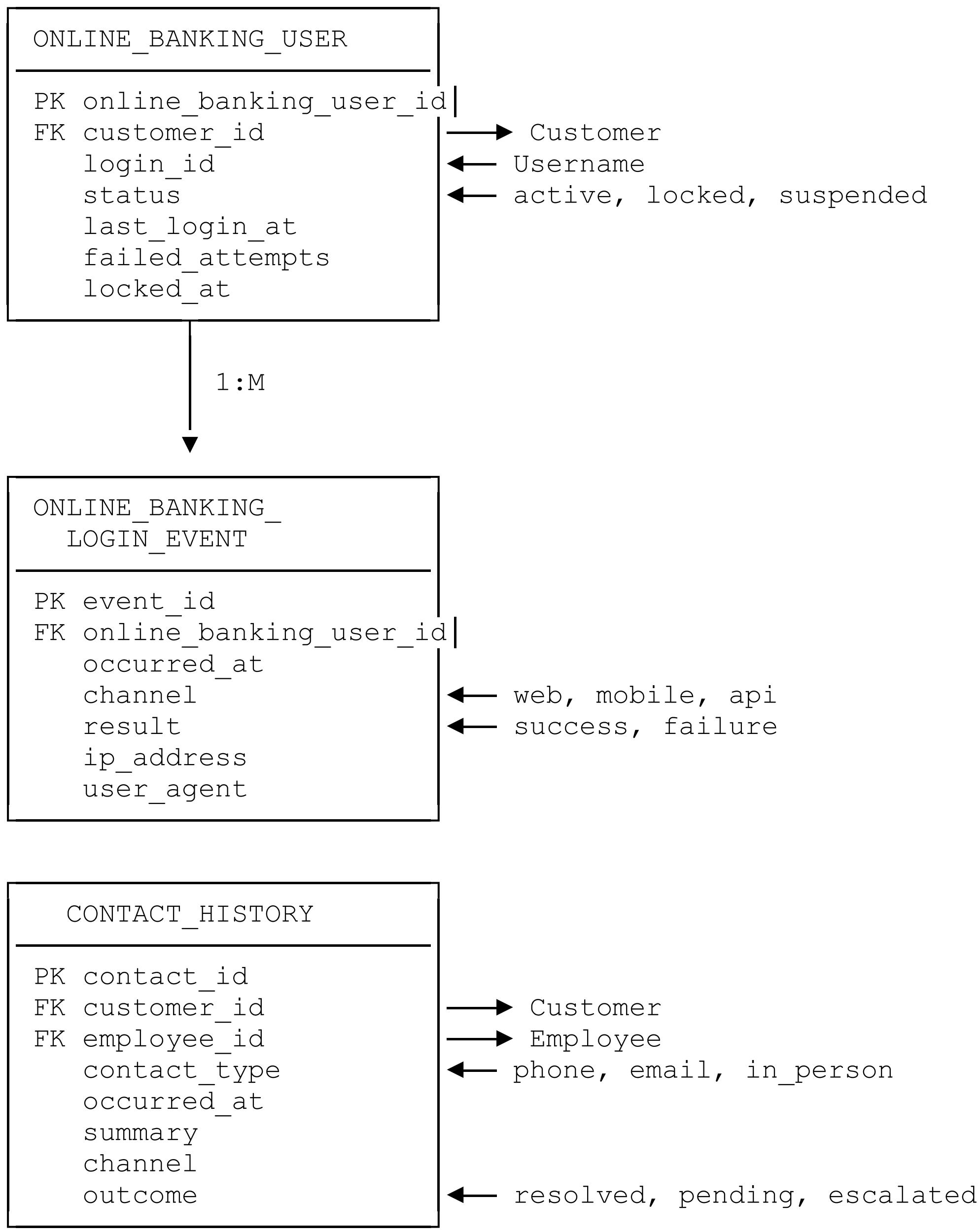
```
ENTITY_CONTACT (Polymorphic)
--------------------------------------------------------------
PK entity_contact_id
   entity_type          <--  "customer", "account", "employee"
   entity_id            <--  References ANY entity
FK contact_id           -->  CONTACT_INFO table
   contact_purpose      <--  "primary", "emergency"
   is_current
```

REFERENCE/LOOKUP TABLES

```
SIC_CODE
----------------
PK sic_code
   description
   is_active

TRANSACTION_
  CATEGORY
----------------
PK category_id
   name
FK parent_id      <--  Hierarchical
   group_code     <--  Dashboard
grouping
```

ONLINE BANKING & REPORTING



Detailed Relationship Cardinality

One-to-Many Relationships

Parent Table	Child Table	Relationship	Description
customer	account_ownership	1:M	One customer can own multiple accounts
account	account_ownership	1:M	One account can have multiple owners
customer	household_membership	1:M	One customer can belong to multiple households
household	household_membership	1:M	One household has multiple members
account	financial_transaction	1:M	One account has many transactions
debit_card	financial_transaction	1:M	One card has many transactions
customer	entity_address	1:M	One customer can have multiple addresses
customer	entity_contact	1:M	One customer can have multiple contacts
branch	employee_branch	1:M	One branch has multiple employees
employee	employee_branch	1:M	One employee can work at multiple branches
debit_card_limit_profile	debit_card	1:M	One profile applied to many cards

Many-to-Many Relationships

Table 1	Junction Table	Table 2	Description
customer	account_ownership	account	Customers own accounts
employee	employee_branch	branch	Employees assigned to branches
customer	household_membership	household	Customers grouped in households
customer	customer_sic_code	sic_code	Customers classified by industry
customer	customer_officer_assignment	officer_code	Customers assigned to officers

Self-Referencing Relationships

Table	Column	Description
transaction_category	parent_id → category_id	Hierarchical categories
financial_transaction	related_transaction_id → transaction_id	Related/linked transactions

---

Business Rule Enforcement Map

Database-Level Constraints

```
┌─────────────────────────────────────────────────────────────────┐
│                    CONSTRAINT ENFORCEMENT                        │
└─────────────────────────────────────────────────────────────────┘

CUSTOMER TABLE:
├─ CHECK: customer_name_type_check
│  └─ Individual customers MUST have first_name + last_name
│  └─ Business customers MUST have business_name
│
├─ TRIGGER: customer_full_name_trigger (PostgreSQL)
│  └─ Auto-populates full_name for unified search
│
└─ COMPUTED: full_name (SQL Server)
   └─ Persisted computed column for search

ACCOUNT TABLE:
└─ FK: sic_code → sic_code table
   └─ Industry classification validation

DEBIT_CARD TABLE:
├─ CHECK: chk_expiry_month (1-12)
├─ CHECK: chk_expiry_year (≥ current year)
│
├─ TRIGGER: trg_validate_debit_card_account_type
│  └─ ENFORCES: Only checking/business_checking accounts can have cards
│  └─ PostgreSQL: BEFORE INSERT/UPDATE trigger
│  └─ SQL Server: FOR INSERT/UPDATE trigger with ROLLBACK
│
└─ FK: account_id → account(account_id)
   └─ Debit card must link to valid account

FINANCIAL_TRANSACTION TABLE:
├─ UNIQUE: (account_id, source_system, source_transaction_id)
│  └─ Prevents duplicate transaction imports
│
└─ FK: debit_card_id → debit_card(card_id) ON DELETE SET NULL
   └─ Optional card linkage (only for card transactions)
```

Index Strategy Map

High-Performance Query Patterns

```
┌─────────────────────────────────────────────────────────────────┐
│                     INDEXING STRATEGY                           │
└─────────────────────────────────────────────────────────────────┘

CUSTOMER SEARCH:
├─ PostgreSQL: GIN index on full_name using pg_trgm
│  └─ Fuzzy/similarity search: WHERE full_name % 'search_term'
│
└─ SQL Server: Full-text index on (first_name, last_name, full_name)
   └─ CONTAINS() for flexible search
   └─ STRING_SIMILARITY() for fuzzy matching (SQL Server 2022)

TRANSACTION QUERIES:
├─ Composite index: (account_id, posting_date, transaction_id)
│  └─ Optimizes: Recent transactions by account
│
├─ Composite index: (account_id, status, posting_date)
│  └─ Optimizes: Status-filtered transaction history
│
└─ Index: (debit_card_id, posting_date DESC)
   └─ Optimizes: Card transaction history

ACCOUNT LOOKUPS:
├─ Unique index: account_number
├─ Index: account_type (for type-specific queries)
└─ Index: jack_henry_account_id (core banking integration)

RELATIONSHIP QUERIES:
├─ Composite index: (employee_id, is_primary) on employee_branch
│  └─ Fast primary branch lookup
│
└─ Composite index: (customer_id, officer_code) on customer_officer
   └─ Customer-officer relationship queries
```

Data Flow Diagrams

Transaction Import Flow

```
┌─────────────────────────────────────────────────────────────────────┐
│            TRANSACTION IMPORT FROM CORE BANKING                     │
└─────────────────────────────────────────────────────────────────────┘

    JACK HENRY / SILVERLAKE
            │ Raw Transaction Data
            ▼
    ┌───────────────────┐
    │ Transaction       │
    │ Import Service    │
    └───────────────────┘
              │
              │ Validate & Transform
              ▼
    ┌───────────────────────────┐
    │ FINANCIAL_TRANSACTION     │
    │───────────────────────────│
    │ • account_id lookup       │
    │ • debit_card_id matching  │
    │ • category_id assignment  │
    │ • balance calculation     │
    │ • deduplication check     │
    └───────────────────────────┘
              │
              │ Unique constraint check
              │ (account_id, source_system, source_transaction_id)
              │
              ├─ DUPLICATE? → Skip/Update
              │
              └─ NEW? → Insert
                     │
                     ▼
           Update account balance
           Update last_transaction_date
```

Customer Search Flow

```
┌─────────────────────────────────────────────────────────────────┐
│                INTELLIGENT CUSTOMER SEARCH                       │
└─────────────────────────────────────────────────────────────────┘

   User Search Input: "john smyth"
          │
          ▼
   ┌──────────────────┐
   │ Search Detection │
   └────────┬─────────┘
            │
            ├─ Is numeric? → Customer ID search
            │                   └─ Exact match (100% score)
            │
            ├─ Contains @?  → Email search
            │                   └─ entity_contact lookup
            │
            └─ Text input  → Fuzzy name search
                               │
                               ▼
                  ┌──────────────────────┐
                  │ PostgreSQL: pg_trgm  │
                  │ SIMILARITY() search  │
                  ├──────────────────────┤
                  │ "Smyth" finds "Smith"│
                  │ Match score: 33%     │
                  └──────────────────────┘
                               │
                  ┌────────────────────┐
                  │ SQL Server 2022:   │
                  │ STRING_SIMILARITY()│
                  ├────────────────────┤
                  │ SQL Server 2019:   │
                  │ SOUNDEX/DIFFERENCE │
                  └────────────────────┘
                               │
                               ▼
                 Return ranked results
                 (match_score DESC)
```

Security & Compliance Architecture

PCI-DSS Compliance Boundaries

```
┌─────────────────────────────────────────────────────────────────┐
│              PCI-DSS CARDHOLDER DATA STORAGE                     │
└─────────────────────────────────────────────────────────────────┘

DEBIT_CARD TABLE:
├─ ✅ ALLOWED (stored):
│  ├─ last_four_digits (last 4 of PAN)
│  ├─ card_brand (Visa, Mastercard)
│  ├─ expiry_month
│  ├─ expiry_year
│  ├─ cardholder_name
│  └─ card tokens (jack_henry_card_id, silverlake_card_token)
│
└─ ❌ PROHIBITED (never stored):
   ├─ Full PAN (Primary Account Number)
   ├─ CVV / CVV2 / CVC2
   ├─ PIN
   └─ Magnetic stripe data

TRANSACTION DATA:
├─ ✅ ALLOWED:
│  ├─ merchant_name
│  ├─ merchant_category_code
│  ├─ amount
│  └─ debit_card_id (reference only)
│
└─ ❌ PROHIBITED:
   └─ Full card numbers in any field
```

Data Privacy & Encryption Requirements

```
APPLICATION-LAYER ENCRYPTION REQUIRED:
├─ customer.tax_identifier (SSN/EIN)
├─ customer.government_id
└─ Sensitive contact_info values

DATABASE-LEVEL PROTECTION:
├─ Row-level security (if enabled)
├─ Column-level encryption (TDE)
└─ Audit logging for sensitive table access
```

Migration Dependency Graph

```
┌─────────────────────────────────────────────────────────────┐
│                  MIGRATION DEPENDENCIES                      │
└─────────────────────────────────────────────────────────────┘

0001: Extensions/Search Setup
  │
  ├──▶ 0002: Branch/Employee Schema Updates
  │     │
  │     └──▶ 0003: Employee-Branch Junction
  │           │
  │           └──▶ 0004: Officer Code Addition
  │                 │
  │                 └──▶ 0005: Remove Old employee_branch_id
  │
  └──▶ 0006: Customer Name Conditional Logic ◀── Critical
        │
        ├──▶ 0007: Customer-Officer Assignment
        │
        ├──▶ 0008: SIC Code Table
        │     │
        │     ├──▶ 0009: Customer-SIC Junction
        │     │
        │     └──▶ 0013: Account SIC FK
        │
        ├──▶ 0010: Customer is_deceased Flag
        │
        └──▶ 0011: Account Optional Fields
              │
              └──▶ 0012: Account Column Fixes
                    │
                    └──▶ 0014: Debit Card Tables ◀── Latest
```

Table Size & Growth Projections

Expected Data Volumes

Table	Typical Row Count	Growth Rate	Notes
customer	10K - 500K	Slow	100-500 new customers/month
account	50K - 2M	Moderate	1-5 accounts per customer
financial_transaction	1M - 100M+	High	10-100K transactions/day
debit_card	20K - 1M	Moderate	1-2 cards per checking account
household	5K - 200K	Slow	30-50% of customers
contact_history	100K - 5M	Moderate	2-5 contacts/customer/year
online_banking_login_event	500K - 50M	High	Multiple logins/day/user

Performance Monitoring Recommendations

```sql
-- PostgreSQL: Monitor table bloat
SELECT schemaname, tablename,
       pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename))
AS size
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;

-- SQL Server: Monitor index fragmentation
SELECT OBJECT_NAME(object_id) AS table_name,
       index_id, avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED')
WHERE avg_fragmentation_in_percent > 30;
```

Summary Statistics

Database Schema Metrics

Metric	Count	Notes
Total Tables	25+	Core + dashboard + reference
Core Banking Tables	12	customer, account, transaction, etc.
Junction Tables	6	Many-to-many relationships
Lookup/Reference Tables	3	SIC codes, transaction categories
Foreign Keys	35+	Referential integrity enforcement
Indexes	60+	Performance optimization
Triggers	2	Business rule enforcement
Check Constraints	4	Data validation
Unique Constraints	8	Duplicate prevention

End of Entity Relationship Diagram Document